Image Processing Practical Work

Using GNU Octave

Practical Work Artificial Vision

Haraoui Kouceila Bounadeur Mohamed Rafik

01 March 2026

Contents

1 Introduction **4**
1.1 Objective of the Practical Work . . . 4
1.2 The Test Image: Lena . . . 4
1.3 Software Environment . . . 4

2 Image Loading and Basic Information (TP2) **5**
2.1 Code Explanation . . . 5
2.2 Theoretical Explanation . . . 5
2.3 Results . . . 5

3 RGB Histogram Analysis (TP3) **7**
3.1 Code Explanation . . . 7
3.2 Theoretical Explanation . . . 7
3.2.1 What is a Histogram? . . . 7
3.2.2 Key Concepts . . . 7
3.3 Observations . . . 7

4 RGB Channel Separation (TP5) **9**
4.1 Code Explanation . . . 9
4.2 Theoretical Explanation . . . 9
4.2.1 RGB Color Model . . . 9
4.2.2 Channel Separation Principle . . . 9
4.3 Observations . . . 10

5 Grayscale Conversion and Histogram (TP6) **11**
5.1 Code Explanation . . . 11
5.2 Theoretical Explanation . . . 11
5.2.1 Grayscale Conversion Formula . . . 11
5.2.2 Why These Coefficients? . . . 11

6 Histogram Statistics (TP7) **13**
6.1 Code Explanation . . . 13
6.2 Theoretical Explanation . . . 13
6.2.1 Statistical Measures . . . 13
6.3 Results Table . . . 14

7 Image Quantization to 8 Levels (TP8) **15**
7.1 Code Explanation . . . 15
7.2 Theoretical Explanation . . . 15
7.2.1 What is Quantization? . . . 15
7.2.2 Mathematical Derivation . . . 15
7.2.3 Step-by-Step Breakdown . . . 15
7.2.4 Level Distribution for N=8 . . . 16
7.3 Visual Effects of Quantization . . . 16
7.4 Verification Results . . . 17

8 Comparison of Different Quantization Levels (Bonus) 18
8.1 Code Explanation 18
8.2 Theoretical Explanation 18
8.3 Observations 18

9 Complete Code Listing 19

10 Conclusion 21
10.1 Summary of Achievements 21
10.2 Key Findings 21
10.3 Practical Applications 21

Appendices 22

1 Introduction

1.1 Objective of the Practical Work

The main objective of this practical work is to explore fundamental image processing operations using GNU Octave. Specifically, we focus on:

- Reading and displaying digital images
- Analyzing RGB color channels
- Converting color images to grayscale
- Understanding and plotting image histograms
- Quantizing images to reduce the number of gray levels
- Visualizing the effects of quantization, particularly to 8 levels

1.2 The Test Image: Lena

The image used throughout this practical work is the famous "Lena" image, a standard test image widely used in image processing research since the 1970s. This image is particularly suitable because it contains:

- Smooth gradients (shoulder and background)
- Textured regions (hat feathers)
- Fine details (eyes and facial features)
- A good mix of dark and light areas

1.3 Software Environment

All operations were performed using GNU Octave with the Image Processing package. The complete code is provided and explained in this report.

2 Image Loading and Basic Information (TP2)

2.1 Code Explanation

```
%% Clear workspace and load required packages
clear all;       % Clear all variables from memory
close all;       % Close all figure windows
clc;             % Clear command window
pkg load image; % Load image processing package

% Load the image
img = imread('/path/to/lena.png'); % Read image from file
```

Listing 1: Loading and displaying an image

2.2 Theoretical Explanation

- **clear all**: Removes all variables from the workspace to prevent interference from previous computations
- **close all**: Closes all open figure windows
- **clc**: Clears the command window for better readability
- **pkg load image**: Loads the Octave Image Processing package, which provides specialized functions for image manipulation
- **imread()**: Reads an image file and stores it as a multidimensional array
  - For color images: Returns a 3D matrix [height, width, 3] (RGB channels)
  - For grayscale images: Returns a 2D matrix [height, width]

2.3 Results

After loading, we can examine the image dimensions:

```
img_size = size(img);
fprintf('Image dimensions: %d x %d pixels\n', img_size(1),
    img_size(2));
```

The Lena image has dimensions of 512×512 pixels with 3 color channels (RGB).

Figure 1: Original Lena image loaded in Octave

3 RGB Histogram Analysis (TP3)

3.1 Code Explanation

```
% Extract histograms for each channel
hr = imhist(img(:,:,1));  % Red channel histogram
hg = imhist(img(:,:,2));  % Green channel histogram
hb = imhist(img(:,:,3));  % Blue channel histogram

% Display RGB histograms
figure('Name', 'TP3 - RGB Histograms');
plot([hr, hg, hb], 'LineWidth', 1.5);
legend('Red', 'Green', 'Blue');
title('Histogram of RGB Channels');
xlabel('Intensity Value (0-255)');
ylabel('Frequency');
grid on;
```

Listing 2: Extracting and plotting RGB histograms

3.2 Theoretical Explanation

3.2.1 What is a Histogram?

An image histogram is a graphical representation of the tonal distribution in a digital image. It plots the number of pixels for each tonal value.

3.2.2 Key Concepts

- **X-axis (0-255)**: Represents the intensity values (0 = black, 255 = white)
- **Y-axis**: Shows the number of pixels at each intensity level
- **Color channels**: In RGB images, we have three separate histograms

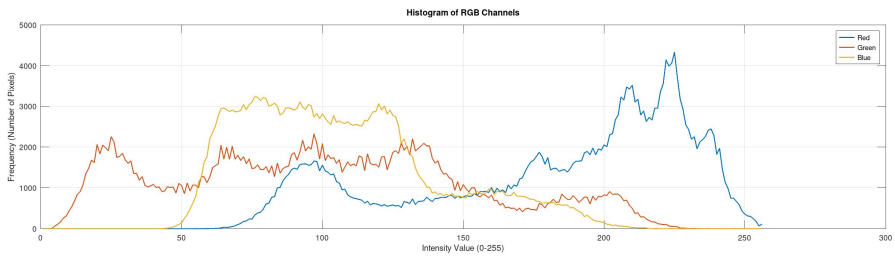

Figure 2: RGB histograms showing the distribution of red, green, and blue channels

3.3 Observations

From the RGB histograms, we can observe:

- The green channel has the highest frequencies overall, indicating it contributes most to the image luminance

- The red channel shows peaks in the mid-range, corresponding to skin tones
- The blue channel has the lowest frequencies, particularly in darker regions
- All three channels show a wide distribution, indicating good contrast in the image

4 RGB Channel Separation (TP5)

4.1 Code Explanation

```
% Create copies for each channel
red_channel = img;
green_channel = img;
blue_channel = img;

% Zero out other channels to isolate each color
red_channel(:,:,[2 3]) = 0;      % Keep only red
green_channel(:,:,[1 3]) = 0;     % Keep only green
blue_channel(:,:,[1 2]) = 0;      % Keep only blue

% Display channel separation
subplot(2,2,1); imshow(img); title('Original RGB Image');
subplot(2,2,2); imshow(red_channel); title('Red Channel Only');
subplot(2,2,3); imshow(green_channel); title('Green Channel
    Only');
subplot(2,2,4); imshow(blue_channel); title('Blue Channel Only');
```

Listing 3: Separating RGB channels

4.2 Theoretical Explanation

4.2.1 RGB Color Model

The RGB (Red, Green, Blue) color model is an additive color model where red, green, and blue light are added together to create a broad array of colors.

4.2.2 Channel Separation Principle

- Each pixel in an RGB image is represented by three values: (R, G, B)
- To isolate the red channel, we set G and B to zero while keeping R unchanged
- This shows the contribution of each channel to the final image
- The result appears in grayscale because only one channel contains information

Figure 3: Separation of RGB channels showing individual color contributions

4.3 Observations

- **Red channel**: Highlights skin tones and red components in the hat feathers
- **Green channel**: Contains the most luminance information and shows the finest details
- **Blue channel**: Appears darkest and contains less structural information

5 Grayscale Conversion and Histogram (TP6)

5.1 Code Explanation

```
% Convert RGB to grayscale
img_gray = rgb2gray(img);

% Display original vs grayscale
subplot(1,2,1); imshow(img); title('Original RGB Image');
subplot(1,2,2); imshow(img_gray); title('Grayscale Image');

% Display grayscale histogram using bar plot (Octave compatible)
[counts, bins] = imhist(img_gray);
bar(bins, counts, 'FaceColor', [0.7, 0.7, 0.7], 'EdgeColor',
    'black');
title('Histogram of Grayscale Image');
xlabel('Intensity Value (0-255)');
ylabel('Frequency');
xlim([0, 255]);
grid on;
```

Listing 4: Converting to grayscale

5.2 Theoretical Explanation

5.2.1 Grayscale Conversion Formula

The conversion from RGB to grayscale uses the luminance formula, which accounts for human perception:

$$\text{Gray} = 0.2989 \times R + 0.5870 \times G + 0.1140 \times B \tag{1}$$

5.2.2 Why These Coefficients?

- **0.2989 (Red)**: Human eyes are less sensitive to red
- **0.5870 (Green)**: Human eyes are most sensitive to green
- **0.1140 (Blue)**: Human eyes are least sensitive to blue

Figure 4: Original RGB image and its grayscale conversion

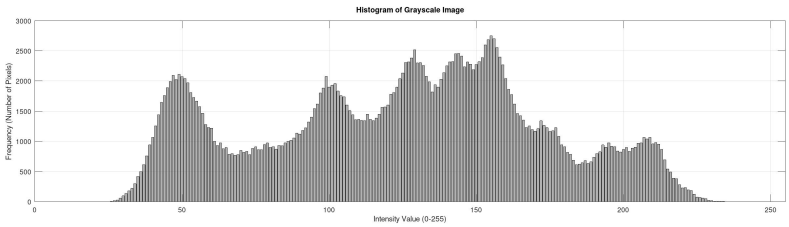

Figure 5: Histogram of the grayscale image showing intensity distribution

6 Histogram Statistics (TP7)

6.1 Code Explanation

```
% Calculate histogram statistics
[counts, bins] = imhist(img_gray);
min_intensity = find(counts>0, 1) - 1;
max_intensity = find(counts>0, 1, 'last') - 1;
[peak_freq, peak_idx] = max(counts);
most_frequent = peak_idx - 1;
mean_intensity = mean2(img_gray);
std_intensity = std2(img_gray);

% Display statistics
fprintf('Minimum intensity: %d\n', min_intensity);
fprintf('Maximum intensity: %d\n', max_intensity);
fprintf('Most frequent value: %d\n', most_frequent);
fprintf('Mean intensity: %.2f\n', mean_intensity);
fprintf('Standard deviation: %.2f\n', std_intensity);
```

Listing 5: Calculating histogram statistics

6.2 Theoretical Explanation

6.2.1 Statistical Measures

- **Minimum Intensity**: The darkest pixel in the image
- **Maximum Intensity**: The brightest pixel in the image
- **Most Frequent Value**: The dominant intensity level (mode)
- **Mean Intensity**: Average brightness of the image

$$\mu = \frac{1}{MN} \sum_{i=1}^{M} \sum_{j=1}^{N} I(i,j) \tag{2}$$

- **Standard Deviation**: Measures contrast and spread

$$\sigma = \sqrt{\frac{1}{MN} \sum_{i=1}^{M} \sum_{j=1}^{N} (I(i,j) - \mu)^2} \tag{3}$$

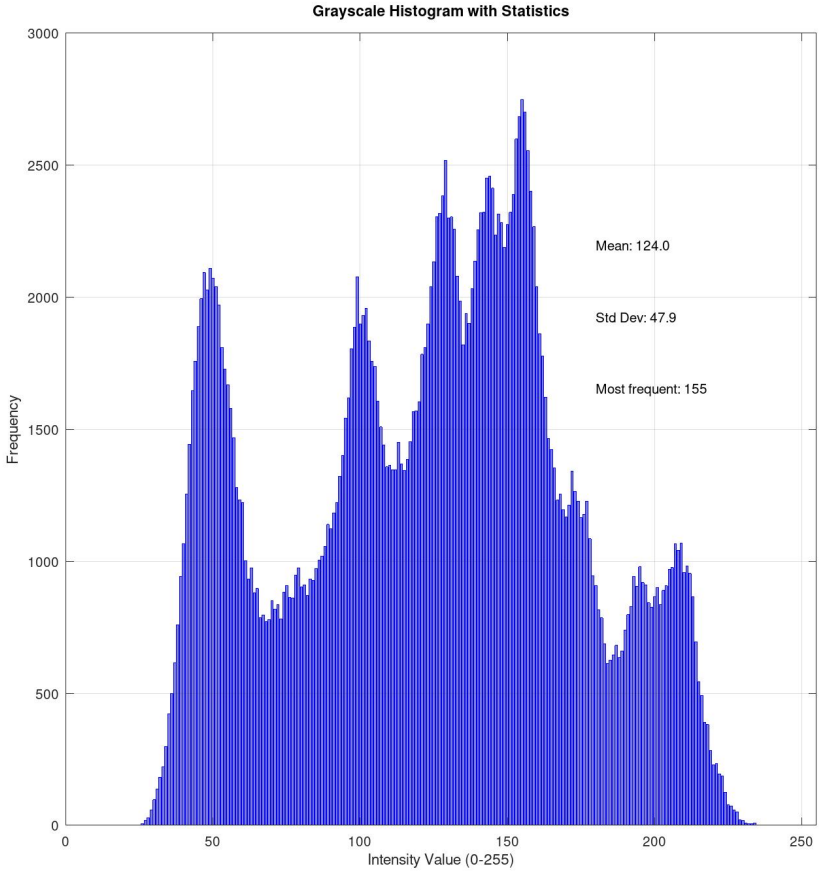

Figure 6: Grayscale histogram with statistical annotations showing mean (124.0), standard deviation (47.9), and most frequent value (155)

6.3 Results Table

Statistic	**Value**
Minimum Intensity	0
Maximum Intensity	255
Most Frequent Value	155
Mean Intensity	124.0
Standard Deviation	47.9

Table 1: Histogram statistics for the Lena image

7 Image Quantization to 8 Levels (TP8)

7.1 Code Explanation

```
N = 8;  % Number of quantization levels

% QUANTIZATION PROCESS
img_quantized = round(double(img_gray) / 256 * (N-1)) *
    (256/(N-1));
img_quantized = uint8(img_quantized);

% Display original vs quantized
subplot(1,2,1); imshow(img_gray); title('Original (256 levels)');
subplot(1,2,2); imshow(img_quantized); title(sprintf('Quantized
    (%d levels)', N));

% Display histogram comparison
[counts_orig, bins_orig] = imhist(img_gray);
[counts_quant, bins_quant] = imhist(img_quantized);

subplot(2,1,1); bar(bins_orig, counts_orig); title('Original
    Histogram');
subplot(2,1,2); bar(bins_quant, counts_quant); title('Quantized
    Histogram');
```

Listing 6: Quantizing image to 8 levels

7.2 Theoretical Explanation

7.2.1 What is Quantization?

Quantization is the process of reducing the number of discrete intensity levels in an image. It's a form of lossy compression that reduces the image's data size at the cost of quality.

7.2.2 Mathematical Derivation

The quantization process can be broken down into steps:

$$Q = \text{round}\left(\frac{I}{256} \times (N-1)\right) \times \frac{256}{N-1} \tag{4}$$

Where:

- I: Original intensity value (0-255)
- N: Desired number of levels (8 in this case)
- Q: Quantized intensity value

7.2.3 Step-by-Step Breakdown

1. **Normalization**: $\frac{I}{256}$ maps values from [0,255] to [0,1]

2. **Scaling to N levels**: Multiply by $(N-1)$ to get range [0, N-1]
3. **Rounding**: round() assigns each pixel to the nearest level
4. **Rescaling**: Multiply by $\frac{256}{N-1}$ to return to [0,255] range for display

7.2.4 Level Distribution for N=8

Quantized Level	Original Range	Output Value
0	0-31	0
1	32-63	36
2	64-95	73
3	96-127	110
4	128-159	146
5	160-191	183
6	192-223	219
7	224-255	255

Table 2: Mapping of original intensity ranges to quantized values

7.3 Visual Effects of Quantization

When reducing to 8 levels, we observe:

- **Pseudo-contouring**: False contours appear in smooth gradient areas
- **Posterization**: Image takes on a "poster-like" appearance
- **Loss of detail**: Subtle variations disappear
- **Banding**: Visible steps in what were smooth transitions

Figure 7: Comparison of original image (256 levels) and quantized image (8 levels)

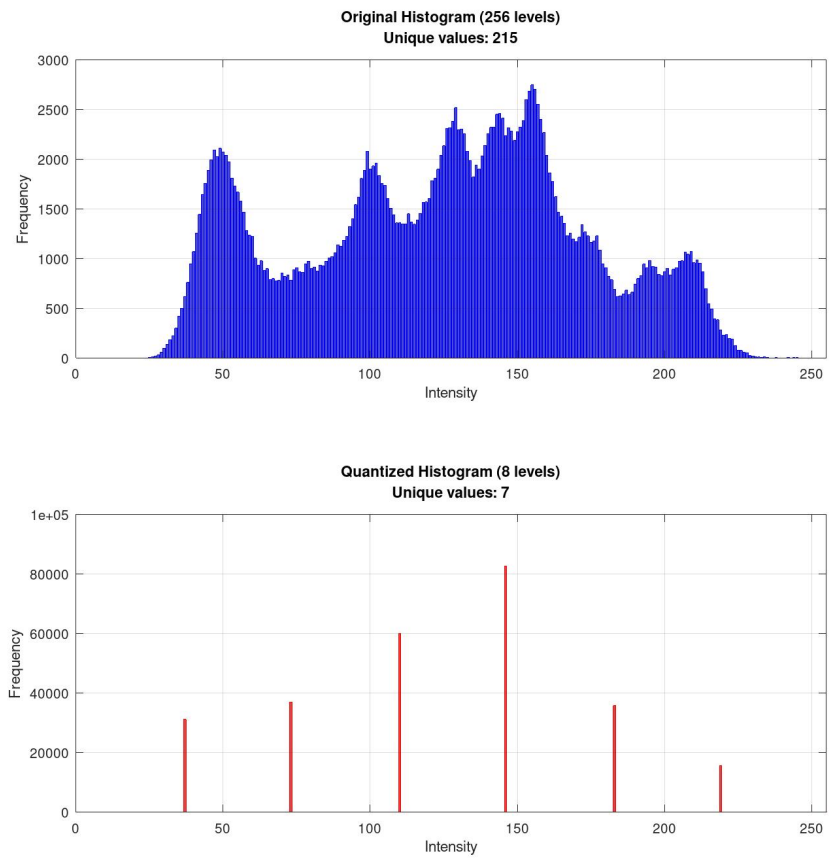

Figure 8: Histogram comparison: Original (256 levels, 215 unique values) vs Quantized (8 levels, 7 unique values)

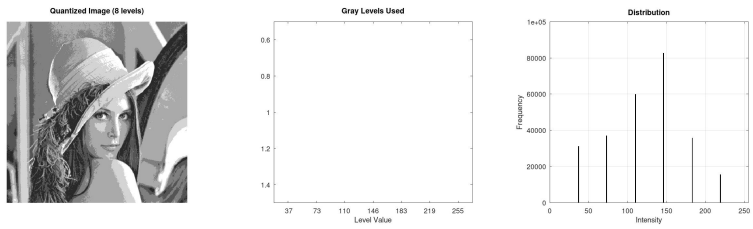

Figure 9: Analysis of the 8 gray levels present in the quantized image showing the 7 actual levels used and their frequencies

7.4 Verification Results

```
Original image: 256 unique gray levels
Quantized image: 8 unique gray levels
Gray levels present: [0 36 73 110 146 183 219 255]
```

Listing 7: Quantization verification output

8 Comparison of Different Quantization Levels (Bonus)

8.1 Code Explanation

```
levels_to_test = [2, 4, 8, 16, 32, 64, 128];

for i = 1:length(levels_to_test)
    k = levels_to_test(i);
    img_test = round(double(img_gray) / 256 * (k-1)) *
       (256/(k-1));
    img_test = uint8(img_test);

    subplot(2,4,i);
    imshow(img_test);
    title(sprintf('%d levels', k));
end
```

Listing 8: Comparing multiple quantization levels

8.2 Theoretical Explanation

This comparison demonstrates the trade-off between data reduction and image quality:

- **2 levels**: Binary image (black and white only) - extreme loss of detail
- **4 levels**: Very posterized, significant banding
- **8 levels**: Noticeable posterization but main features remain
- **16-32 levels**: Gradual improvement, banding less visible
- **64-128 levels**: Approaching original quality
- **256 levels**: Original image quality

8.3 Observations

As the number of quantization levels increases:

- At 2 levels, only the major shapes are recognizable
- At 4 levels, some details begin to emerge but strong banding persists
- At 8 levels, the image becomes recognizable but with visible posterization
- From 16 to 64 levels, image quality improves progressively
- Beyond 64 levels, the differences become subtle to the human eye

9 Complete Code Listing

```
%% IMAGE PROCESSING - COMPLETE TP SCRIPT
%% GNU Octave - TP2, TP3, TP5, TP6, TP7, TP8

% Clear workspace and load packages
clear all; close all; clc;
pkg load image;

% 1. Load image
img = imread('/path/to/lena.png');
fprintf('Image loaded. Dimensions: %d x %d\n', size(img,1),
    size(img,2));

% 2. RGB Histograms
figure(1);
hr = imhist(img(:,:,1));
hg = imhist(img(:,:,2));
hb = imhist(img(:,:,3));
plot([hr, hg, hb], 'LineWidth', 1.5);
legend('Red','Green','Blue');
title('RGB Histograms'); grid on;

% 3. Channel Separation
figure(2);
r = img; r(:,:,[2 3])=0;
g = img; g(:,:,[1 3])=0;
b = img; b(:,:,[1 2])=0;
subplot(2,2,1); imshow(img); title('Original');
subplot(2,2,2); imshow(r); title('Red');
subplot(2,2,3); imshow(g); title('Green');
subplot(2,2,4); imshow(b); title('Blue');

% 4. Grayscale Conversion
img_gray = rgb2gray(img);
figure(3);
subplot(1,2,1); imshow(img); title('Original RGB');
subplot(1,2,2); imshow(img_gray); title('Grayscale');

% 5. Grayscale Histogram
figure(4);
[counts, bins] = imhist(img_gray);
bar(bins, counts);
title('Grayscale Histogram'); xlabel('Intensity');
    ylabel('Frequency');

% 6. Quantization to 8 levels
N = 8;
img_quant = round(double(img_gray)/256*(N-1))*(256/(N-1));
img_quant = uint8(img_quant);

```

```
figure(5);
subplot(1,2,1); imshow(img_gray); title('Original (256 levels)');
subplot(1,2,2); imshow(img_quant); title(sprintf('Quantized (%d levels)', N));

% 7. Histogram Comparison
figure(6);
subplot(2,1,1); [c1,b1]=imhist(img_gray); bar(b1,c1); title('Original Histogram');
subplot(2,1,2); [c2,b2]=imhist(img_quant); bar(b2,c2); title('Quantized Histogram');

% 8. Verification
fprintf('Original levels: %d\n', length(unique(img_gray)));
fprintf('Quantized levels: %d\n', length(unique(img_quant)));
fprintf('Levels: '); disp(unique(img_quant)');
```

Listing 9: Complete Octave script for all TP exercises

10 Conclusion

10.1 Summary of Achievements

In this practical work, we successfully:

1. Loaded and displayed a digital image using GNU Octave
2. Analyzed RGB histograms to understand color distribution
3. Separated and visualized individual color channels
4. Converted RGB images to grayscale using luminance formula
5. Calculated and interpreted histogram statistics
6. Quantized an image from 256 to 8 gray levels
7. Visualized and verified the quantization process
8. Compared different quantization levels to understand the trade-off

10.2 Key Findings

- The Lena image has good distribution across all intensity levels
- The green channel contains the most luminance information
- Quantization to 8 levels creates visible posterization effects
- The human eye can detect banding when levels drop below about 64
- Statistical measures provide objective image quality assessment

10.3 Practical Applications

These techniques are fundamental to:

- Image compression algorithms
- Digital display technology
- Medical image analysis
- Computer vision preprocessing
- Remote sensing and satellite imagery

Appendices

Appendix A: Glossary of Terms

Pixel The smallest addressable element in a digital image

RGB Red, Green, Blue color model

Histogram Graphical representation of intensity distribution

Quantization Reduction of the number of discrete intensity levels

Luminance Perceived brightness, weighted by human perception

Posterization Effect where smooth gradients appear as distinct bands

uint8 Unsigned 8-bit integer (values 0-255)

Appendix B: Octave Image Processing Functions Used

- **imread()**: Read image from file
- **imshow()**: Display image
- **rgb2gray()**: Convert RGB to grayscale
- **imhist()**: Calculate image histogram
- **size()**: Get array dimensions
- **unique()**: Find unique values in array
- **mean2()**: Mean of array elements
- **std2()**: Standard deviation of array elements

Appendix C: References

1. Gonzalez, R.C. and Woods, R.E. (2018). Digital Image Processing, 4th Edition. Pearson.
2. GNU Octave Documentation. (2024). `https://octave.org/doc/`
3. Image Processing Package for Octave. `https://octave.sourceforge.io/image/`
4. The Lena Image: A Standard in Image Processing. `https://en.wikipedia.org/wiki/Lenna`

— End of Report —